INFO B302 - Introduction à la démarche scientifique

Professeur : Xavier Devroey <xavier.devroey@unamur.be>
Assistants : Jérôme Maquoi <jerome.maquoi@unamur.be>

Contextes et sujets de recherche

Table des matières

1 Contextes et sujets de recherche **2**
1.1 Continuous integration pipeline 2
1.2 Automated test case generation adoption in practice 2
1.3 Search-based software testing 3
1.4 Fault localization and debugging 4
1.5 Flaky test detection and debugging 5
1.6 Socio-technical debt 6
1.7 Testing machine-learning-based systems 6
1.8 Machine learning for software engineering 7
1.9 Mutation testing 8
1.10 Software testing education 9
1.11 Diversity and inclusion in computer science 9
1.12 Green computing 10

---

1 Contextes et sujets de recherche

Dans le cadre du travail fourni durant le cours, le **contexte de la recherche** est à choisir parmi l'une des thématiques suivantes. Ces thématiques proposent une description générale d'un domaine de recherche, des exemples de sujets de recherche (*topics of interest*) ainsi qu'une courte liste de références représentatives permettant de se faire une idée du type de recherche menées.

Chaque groupe doit choisir un et un seul sujets de recherche (*topics of interest*).

1.1 Continuous integration pipeline

Continuous integration (CI) pipelines are a common way to automate several tasks during software development. For instance, CI can be used to build (compile, test, and package) applications based on some specific events (like a push in a Git repository). Several technologies have been developed around CIs (Jenkins, Travis, etc.) and successfully deployed in industry. Nevertheless, with the rapid progress made in software development activities, CI pipelines have become more and more complex to handle specific aspects, like security, advanced code analysis, automated testing, dependency management, etc.

Examples of topics of interest

- Design of complex CI pipelines : identifying requirements for complex pipelines, modelisation of CI pipelines, handling CI configurations, etc.
- Validation of complex CI pipelines : static analysis of CI pipelines configurations, testing of CI pipelines, etc.
- Visualization and monitoring of complex CI pipelines
- Current practices around CI : good and bad practices in open source/industry projects, adoption of CI by professional developers, etc.

Relevant publications

- Hilton, M., Tunnell, T., Huang, K., Marinov, D. and Dig, D. 2016. Usage, costs, and benefits of continuous integration in open-source projects. ASE 2016 - Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering. (2016), 426–437. `https://doi.org/10.1145/2970276.2970358`.
- Jin, X. and Servant, F. 2021. What Helped, and what did not ? An Evaluation of the Strategies to Improve Continuous Integration. 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE) (May 2021), 213–225. `https://doi.org/10.1109/ICSE43902.2021.00031`.
- Jin, X. and Servant, F. 2020. A cost-efficient approach to building in continuous integration. Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (Jun. 2020), 13–25. `https://doi.org/10.1145/3377811.3380437`.
- Memon, A., Zebao Gao, Bao Nguyen, Dhanda, S., Nickell, E., Siemborski, R. and Micco, J. 2017. Taming Google-scale continuous testing. 2017 IEEE/ACM 39th International Conference on Software Engineering : Software Engineering in Practice Track (ICSE-SEIP) (May 2017), 233–242. `https://doi.org/10.1109/ICSE-SEIP.2017.16`.
- Spieker, H., Gotlieb, A., Marijan, D. and Mossige, M. 2017. Reinforcement learning for automatic test case prioritization and selection in continuous integration. Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (Jul. 2017), 12–22. `https://doi.org/10.1145/3092703.3092709`.

1.2 Automated test case generation adoption in practice

An essential step to detect failures prior to software deployment consists in testing : the systematic execution of test cases on a Software Under Test (SUT) to check that it behaves as expected. Test cases can be manually written or automatically generated, either from a specification defining valid behaviors of the software (e.g., in a behavioral model) or directly from the source code using a search-based approach. The former requires the developer to design a

(relatively complete) model of the SUT, making the approach unaffordable for large systems. The latter can directly be applied to the SUT source code but, without any additional information, cannot automatically decide if one execution of the SUT, triggered by a test case, is valid or not).

Examples of topics of interest

- Adoption of model-based testing in practice : current practices around model-based testing in industry, study of model-based practices across different application domains (embedded systems, BDD developments, safety critical systems, etc.), model-based testing in open-source projects, case study on the adoption of model-based testing, etc.
- Adoption of search-based test case generation in practice : study of search-based test case generation across different application domains (unit testing of Java applications, testing REST APIs, testing self-driving cars, testing embedded systems, software product lines, etc.), case study on the adoption of search-based test case generation, etc.

Relevant publications

- Ali, S., Yue, T., Briand, L. and Walawege, S. 2012. A Product Line Modeling and Configuration Methodology to Support Model-Based Testing : An Industrial Case Study. Model Driven Engineering Languages and Systems. MODELS 2012. (2012), 726–742. `https://doi.org/10.1007/978-3-642-33666-9_46`.
- Arcuri, A. 2018. An experience report on applying software testing academic results in industry : we need usable automated test generation. Empirical Software Engineering. 23, 4 (Aug. 2018), 1959–1981. `https://doi.org/10.1007/s10664-017-9570-9`.
- Almasi, M.M., Hemmati, H., Fraser, G., Arcuri, A. and Benefelds, J. 2017. An Industrial Evaluation of Unit Test Generation : Finding Real Faults in a Financial Application. 2017 IEEE/ACM 39th International Conference on Software Engineering : Software Engineering in Practice Track (ICSE-SEIP) (May 2017), 263–272. `https://doi.org/10.1109/ICSE-SEIP.2017.27`.

1.3 Search-based software testing

Search-based software testing designates a set of AI techniques relying on evolutionary computation (i.e., search-based approaches) to address software testing challenges. Those challenges cover various testing activities, like test case generation, test case selection (which test should be executed to find as many bugs as possible ?) and prioritization (how should test cases be ordered to find bugs as fast as possible ?), etc. Evolutionary computation algorithms rely on the idea (coming from the theory of evolution) that a population of candidate solutions (test suites, a subset of test to execute, ordered lists of tests, etc.) will evolve and improve over time. For instance, genetic algorithms will use reproduction among the individuals of the population to (try to) combine the best characteristics of the different individuals to evolve a population where only the fittest survive.

Examples of topics of interest

- Definition, application, and evaluation of search-based test case generation, including search-based unit test generation, search-based crash reproduction, search-based integration testing, etc.
- Improvement of search-based test case generation approaches to enhance various non-functional aspects (like, readability).
- Definition of test case selection and prioritization approaches, based on various available sources of information (past issues and faults fixed in the source code, characteristics of the source code, etc.).

Relevant publications

— Fraser, G. and Arcuri, A. 2014. A Large-Scale Evaluation of Automated Unit Test Generation Using EvoSuite. ACM Transactions on Software Engineering and Methodology. 24, 2 (Dec. 2014), 1–42. https://doi.org/10.1145/2685612.
— Arcuri, A. 2019. RESTful API Automated Test Case Generation with EvoMaster. ACM Transactions on Software Engineering and Methodology. 28, 1 (Feb. 2019), 1–37. https://doi.org/10.1145/3293455.
— Kechagia, M., Devroey, X., Panichella, A., Gousios, G. and van Deursen, A. 2019. Effective and efficient API misuse detection via exception propagation and search-based testing. Proceedings of the 28th ACM SIGSOFT International Symposium on Software Test-ing and Analysis (Beijing, China, Jul. 2019), 192–203. https://doi.org/10.1145/3293882.3330552.
— Panichella, A., Kifetew, F.M. and Tonella, P. 2018. A large scale empirical comparison of state-of-the-art search-based test case generators. Information and Software Technology. 104, June (2018), 236–256. https://doi.org/10.1016/j.infsof.2018.08.009.
— Derakhshanfar, P., Devroey, X., Perrouin, G., Zaidman, A. and Deursen, A. 2020. Search-based crash reproduction using behavioural model seeding. Software Testing, Verification and Reliability. 30, 3 (May 2020), e1733. https://doi.org/10.1002/stvr.1733.
— Parejo, J.A., Sánchez, A.B., Segura, S., Ruiz-Cortés, A., Lopez-Herrejon, R.E. and Egyed, A. 2016. Multi-objective test case prioritization in highly configurable systems : A case study. Journal of Systems and Software. 122, (2016), 287–310. https://doi.org/10.1016/j.jss.2016.09.045.
— Noguchi, T., Washizaki, H., Fukazawa, Y., Sato, A. and Ota, K. 2015. History-Based Test Case Prioritization for Black Box Testing Using Ant Colony Optimization. 2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST) (Apr. 2015), 1–2. https://doi.org/10.1109/ICST.2015.7102622.

1.4 Fault localization and debugging

Software may fail during its operations, which will (usually) lead to the creation of an entry in an issue (also sometime called ticket) tracking system. Based on the issues, the development team has to fix the faults causing the runtime failures. During this process, called debugging, the developer will try to isolate and reproduce the failure in order to diagnose the problem and identify and fix the underlying fault in the source code. Several activities might be involved in this process : first, the issues have to be sorted (triage) to remove the noise (i.e., false positives) and prioritize the failures to fix. Second, the underlying fault has to be localized in the source code. And finally, the debugging process identifies the root cause and fix the code accordingly.

Examples of topics of interest

— Fault triage in an open-source or industrial application, including the assignment of the faults to fix to the most relevant developer (for instance, the one that has enough knowledge about the part of the code involved in the failure).
— Automated fault localization using various approaches like spectrum-based fault localization, etc.
— Debugging of the source code to identify the root causes of the fault using various techniques like delta-debugging, etc.
— Deployment of automated fault localization in the industry.
— Design, implementation, and evaluation of debuggers relying on delta-debugging, etc.
— Design, implementation, and evaluation of new visualization methods for debugging.
— Development of a theory explaining how faults are introduced in software.

Relevant publications

— Wong, W.E., Gao, R., Li, Y., Abreu, R. and Wotawa, F. 2016. A survey on software fault localization. IEEE Transactions on Software Engineering. 42, 8 (2016), 707–740. https://doi.org/10.1109/TSE.2016.2521368.
— Perez, A., Abreu, R. and van Deursen, A. 2017. A Test-Suite Diagnosability Metric for Spectrum-Based Fault Localization Approaches. 2017 IEEE/ACM 39th International Conference on Soft-

ware Engineering (ICSE) (Buenos Aires, Argentina, May 2017), 654–664. https://doi.org/10.1109/ICSE.2017.66.
— Jones, J.A., Harrold, M.J. and Stasko, J. 2002. Visualization of test information to assist fault localization. Proceedings of the 24th international conference on Software engineering - ICSE '02 (2002), 467. https://doi.org/10.1145/581339.581397.
— Xuan, J. and Monperrus, M. 2014. Test case purification for improving fault localization. Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering - FSE 2014 (2014), 52–63. https://doi.org/10.1145/2635868.2635906.
— Rodríguez-Pérez, G., Robles, G., Serebrenik, A., Zaidman, A., Germán, D.M. and Gonzalez-Barahona, J.M. 2020. How bugs are born : a model to identify how bugs are introduced in software components. Empirical Software Engineering. 25, 2 (Mar. 2020), 1294–1340. https://doi.org/10.1007/s10664-019-09781-y.
— Tomassi, D.A., Dmeiri, N., Wang, Y., Bhowmick, A., Liu, Y.-C., Devanbu, P.T., Vasilescu, B. and Rubio-Gonzalez, C. 2019. BugSwarm : Mining and Continuously Growing a Dataset of Reproducible Failures and Fixes. 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE) (May 2019), 339–349. https://doi.org/10.1109/ICSE.2019.00048.

1.5 Flaky test detection and debugging

Flaky tests are tests which, when re-executed multiple time, provide different outcomes (i.e., sometimes a same test passes, and sometimes it fails). Flaky tests are problematic as they will fail software builds randomly. They have been recognized as a growing problem in the software industry and a lot of effort are currently deployed to identify and fix such tests. The standard approach for identifying flaky tests is to re-execute the same test suite multiple times and monitor the outcomes to detect any difference. This is of course very costly and not scalable to large applications.

Examples of topics of interest

— Early detection of flaky tests, based on the content of the test and infrastructure of the building system.
— Debugging of flaky tests to avoid random build failures.
— Study of the nature of flaky tests (some are due to the infrastructures, other are due to the nature of the test, etc.).
— Commonality of flaky tests in open-source projects.
— Commonality of flaky tests in the industry and how does the industry deal with them.

Relevant publications

— Rehman, M.H.U. and Rigby, P.C. 2021. Quantifying no-fault-found test failures to prioritize inspection of flaky tests at Ericsson. Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Aug. 2021), 1371–1380. https://doi.org/10.1145/3468264.3473930.
— Zhang, P., Jiang, Y., Wei, A., Stodden, V., Marinov, D. and Shi, A. 2021. Domain-Specific Fixes for Flaky Tests with Wrong Assumptions on Underdetermined Specifications. 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE) (May 2021), 50–61. https://doi.org/10.1109/ICSE43902.2021.00018.
— Haben, G., Habchi, S., Papadakis, M., Cordy, M. and Le Traon, Y. 2021. A Replication Study on the Usability of Code Vocabulary in Predicting Flaky Tests. 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR) (May 2021), 219–229. https://doi.org/10.1109/MSR52588.2021.00034.
— Pinto, G., Miranda, B., Dissanayake, S., D'Amorim, M., Treude, C. and Bertolino, A. 2020. What is the Vocabulary of Flaky Tests ? Proceedings of the 17th International Conference on Mining Software Repositories (Jun. 2020), 492–502. https://doi.org/10.1145/3379597.3387482.
— Dong, Z., Tiwari, A., Yu, X.L. and Roychoudhury, A. 2021. Flaky test detection in Android via event order exploration. Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Aug. 2021), 367–378. https://doi.org/10.1145/3468264.3468584.

— Rahman, M.T. and Rigby, P.C. 2018. The impact of failing, flaky, and high failure tests on the number of crash reports associated with Firefox builds. Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Oct. 2018), 857–862. https://doi.org/10.1145/3236024.3275529.

1.6 Socio-technical debt

Technical debt is a metaphor used to describe the impact of bad implementation choices done, generally, to save time. It quantifies the cost of additional rework caused by choosing an easy (limited) solution now instead of using a better approach that would take longer. For instance, duplicating code to save time instead of designing a clean API. Recently, the notion of technical debts has been extended to take social factors, like developers' turnover or development team management into account. Managing socio-technical debt is a key concern for software development teams. Indeed, the higher the technical-debt, the more costly and risky are the future evolutions of the system.

Examples of topics of interest

— Current code knowledge sharing approaches in place in a development team as well as the deployment of such approaches in teams that do not have such processes in place.
— Establishing and studying the relation between social and technical factors : e.g., the impact of the development team organization on the code quality, the impact of developers' turnover on the quality of the source code, etc.
— The design and (efficient) computation of socio-technical indicators used to compute the technical debt.
— Visualization techniques for socio-technical debt for a single developer, a single project, a development team, a project portfolio, etc.

Relevant publications

— Kruchten, P., Nord, R.L. and Ozkaya, I. 2012. Technical debt : From metaphor to theory and practice. IEEE Software. 29, 6 (2012), 18–21. https://doi.org/10.1109/MS.2012.167.
— Tamburri, D.A., Kruchten, P., Lago, P. and Vliet, H. van 2015. Social debt in software engineering : insights from industry. Journal of Internet Services and Applications. 6, 1 (Dec. 2015), 10. https://doi.org/10.1186/s13174-015-0024-6.
— Sviridov, N., Evtikhiev, M. and Kovalenko, V. 2021. TNM : A Tool for Mining of Socio-Technical Data from Git Repositories. 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR) (May 2021), 295–299. https://doi.org/10.1109/MSR52588.2021.00041.
— Falcao, F., Barbosa, C., Fonseca, B., Garcia, A., Ribeiro, M. and Gheyi, R. 2020. On Relating Technical, Social Factors, and the Introduction of Bugs. SANER 2020 - Proceedings of the 2020 IEEE 27th International Conference on Software Analysis, Evolution, and Reengineering. (2020), 378–388. https://doi.org/10.1109/SANER48275.2020.9054824.
— Sousa, A., Rocha, L., Britto, R., Gong, Z. and Lyu, F. 2021. Technical Debt in Large-Scale Distributed Projects : An Industrial Case Study. 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER) (Mar. 2021), 590–594. https://doi.org/10.1109/SANER50967.2021.00071.

1.7 Testing machine-learning-based systems

With the development of machine learning approaches, a large variety of systems rely on ML-trained models to operate. For instance, recommender systems, autonomous systems like self-driving cars, systems using image recognition, etc. However, as any other piece of software ML-based system might contain faults, triggering undesired effects during operations of the system. Such effects can have disastrous effects, like car accidents, setting up biases in hiring process, etc. This context explores how verification and validation approaches, such as testing,

can be applied to ML-based systems to guarantee specific properties (safety, fairness, etc.) of the system.

Examples of topics of interest

- Bias detection and prevention in ML-based systems.
- Design and definition of safety properties for ML-based systems and how to apply them in practice.
- Testing of trained model to guarantee specific safety properties.
- Studying existing approaches to ML-based system verification and validation in an open-source or an industrial context.
- Verification and validation and ML infrastructures, like ML pipelines, ML libraries, etc.

Relevant publications

- Sharma, A. and Wehrheim, H. 2019. Testing Machine Learning Algorithms for Balanced Data Usage. 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST) (Apr. 2019), 125–135. `https://doi.org/10.1109/ICST.2019.00022`.
- Aggarwal, A., Lohia, P., Nagar, S., Dey, K. and Saha, D. 2019. Black box fairness testing of machine learning models. Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Aug. 2019), 625–635. `https://doi.org/10.1145/3338906.3338937`.
- Gambi, A., Mueller, M. and Fraser, G. 2019. Automatically testing self-driving cars with search-based procedural content generation. Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (Jul. 2019), 318–328. `https://doi.org/10.1145/3293882.3330566`.
- Herbold, S. and Haar, T. 2022. Smoke testing for machine learning : simple tests to discover severe bugs. Empirical Software Engineering. 27, 2 (Mar. 2022), 45. `https://doi.org/10.1007/s10664-021-10073-7`.

1.8 Machine learning for software engineering

Several machine learning approaches have been designed and applied to software engineering tasks, including requirements engineering, development, testing, and maintenance. For instance, GitHub recently launched his Copilot IDE plugin to enhance source code autocompletion. Most of those approaches consist in configuring ML algorithms to produce models able to help in some specific tasks. For instance, machine learning for requirements or test case selection and prioritization, fault localization, performance prediction, etc. Developing such approaches faces several challenges, including the adaptation of ML algorithms and problem representation to source code, collection and processing of data, deployment and maintenance of the models, etc.

Examples of topics of interest

- Application of ML techniques to source code, like natural language processing. Such adaptation usually requires revising the data representation to ensure a proper training.
- Application of ML techniques to software testing. For instance, to predict mutation analysis outcomes and avoid re-execution of the test suite, test case prioritization, test case generation, etc.
- Application of unsupervised learning and data visualization to software engineering repositories to explore various aspects of software development processes.

Relevant publications

- Pan, R., Bagherzadeh, M., Ghaleb, T.A. and Briand, L. 2021. Test Case Selection and Prioritization Using Machine Learning : A Systematic Literature Review. (2021), 1–43. `https://doi.org/10.1007/s10664-021-10066-6`.

— Cao, J., Yang, S., Jiang, W., Zeng, H., Shen, B. and Zhong, H. 2020. BugPecker : locating faulty methods with deep learning on revision graphs. Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (Virtual Event, Australia, Dec. 2020), 1214–1218. https://doi.org/10.1145/3324884.3418934.
— Hindle, A., Barr, E.T., Gabel, M., Su, Z. and Devanbu, P. 2016. On the naturalness of software. Communications of the ACM. 59, 5 (2016), 122–131. https://doi.org/10.1145/2902362.
— Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D. and Zhou, M. 2020. CodeBERT : A Pre-Trained Model for Programming and Natural Languages. Findings of the Association for Computational Linguistics : EMNLP 2020 (Stroudsburg, PA, USA, 2020), 1536–1547. https://doi.org/10.18653/v1/2020.findings-emnlp.139.
— Pradel, M. and Sen, K. 2018. DeepBugs : a learning approach to name-based bug detection. Proceedings of the ACM on Programming Languages. 2, OOPSLA (Oct. 2018), 1–25. https://doi.org/10.1145/3276517.

1.9 Mutation testing

Over the years, several techniques (like lines or branch coverage) have been developed to allow developers to check the quality of their tests. Among those, mutation testing injects defects (using mutation operators) to a given source code to see if the current tests can detect them. The approach relies on the competent programmer hypothesis, stating that a programmer will mostly do small mistakes that can cause faults, and the coupling effect, stating that small faults are tightly coupled with more important faults and that discovering the first will also expose the latter. Those two hypotheses have been long debated by the research community (and continue to be). Mutation testing has also been used as a proxy to evaluate several software testing approaches in research. From an industrial perspective, mutation testing helps developers improving their test suites by refining their tests to detect more mutants (for instance, by writing better assertions).

Examples of topics of interest

— Mutation operators design and evaluation for non-functional testing, like performance testing, security testing, etc.
— Reduction and prioritization of the number of mutants to use to reduce the overall execution time of the mutation analysis.
— Applying machine learning technique to reduce the execution time by predicting the outcomes of the mutation testing analysis.
— Visualization of mutation analysis results.
— Using mutation testing to help developers refine the assertions of their tests.

Relevant publications

— Zhang, J., Zhang, L., Harman, M., Hao, D., Jia, Y. and Zhang, L. 2018. Predictive Mutation Testing. IEEE Transactions on Software Engineering. (2018), 1–1. https://doi.org/10.1109/TSE.2018.2809496.
— Vera-Pérez, O.L., Danglot, B., Monperrus, M. and Baudry, B. 2019. A comprehensive study of pseudo-tested methods. Empirical Software Engineering. 24, 3 (Jun. 2019), 1195–1225. https://doi.org/10.1007/s10664-018-9653-2.
— Kintis, M., Papadakis, M., Papadopoulos, A., Valvis, E., Malevris, N. and Le Traon, Y. 2017. How effective are mutation testing tools ? An empirical analysis of Java mutation testing tools with manual analysis and real faults. Empirical Software Engineering. (2017), 1–38. https://doi.org/10.1007/s10664-017-9582-5.
— Chen, Y., Poskitt, C.M. and Sun, J. 2018. Learning from Mutants : Using Code Mutation to Learn and Monitor Invariants of a Cyber-Physical System. Proceedings - IEEE Symposium on Security and Privacy. 2018-May, (2018), 648–660. https://doi.org/10.1109/SP.2018.00016.
— Cornejo Olivares, O.E., Pastore, F. and Briand, L. 2021. Mutation Analysis for Cyber-Physical Systems : Scalable Solutions and Results in the Space Domain. IEEE Transactions on Software Engineering. (Jan. 2021), 1–1. https://doi.org/10.1109/TSE.2021.3107680.

— Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Traon, Y. Le and Harman, M. 2019. Mutation Testing Advances : An Analysis and Survey. Advances in Computers. 275–378. https://doi.org/10.1016/bs.adcom.2018.03.015.

1.10 Software testing education

Systematically testing an application is not an easy task as it requires a mind switch from the developer. Therefore, software testing education is a research topic that has gained a lot of interest in the last years. Several approaches have been proposed to introduce the topic in various curricula at the bachelor or master level, including, using project-based education, gamification, automated evaluation tools, etc.

Examples of topics of interest

— Automated test case analysis for software testing education with hints for students to help them enhance their test suites.
— Gamification of software testing to support motivation of the students.
— Introduction of software testing in project-based learning.

Relevant publications

— Clegg, B.S., Rojas, J.M. and Fraser, G. 2017. Teaching software testing concepts using a mutation testing game. Proceedings - 2017 IEEE/ACM 39th International Conference on Software Engineering : Software Engineering and Education Track, ICSE-SEET 2017. (2017), 33–36. https://doi.org/10.1109/ICSE-SEET.2017.1.
— Garousi, V., Rainer, A., Lauvås, P. and Arcuri, A. 2020. Software-testing education : A systematic literature mapping. Journal of Systems and Software. 165, (2020). https://doi.org/10.1016/j.jss.2020.110570.
— Lazzarini Lemos, O.A., Fagundes Silveira, F., Cutigi Ferrari, F. and Garcia, A. 2016. The impact of Software Testing education on code reliability : An empirical assessment. Journal of Systems and Software. 137, (2016), 497–511. https://doi.org/10.1016/j.jss.2017.02.042.
— Clarke, P.J., Davis, D.L., Chang-Lau, R. and King, T.M. 2017. Impact of Using Tools in an Undergraduate Software Testing Course Supported by WReSTT. ACM Transactions on Computing Education. 17, 4 (Sep. 2017), 1–28. https://doi.org/10.1145/3068324.

1.11 Diversity and inclusion in computer science

Diversity at work can impact performance, financial results, but also the well-being of employees. However, in IT, although doors are gradually opening to women, they generally experience a feeling of isolation and must continually prove their ability, particularly to male colleagues/employers. And they are not the only ones to suffer : our society remains marked by gender stereotypes and heterosexism.

Examples of topics of interest

— Gender bias and stereotypes in computer science education, at all levels of education (primary/secondary/higher) : identification, impacts and/or solutions to remedy them.
— Bias and gender stereotypes in development teams and their impacts on well-being at work.
— Taking the gender dimension into account in software development (non-gender neutral) : differences in approaches to writing and debugging software code.

Relevant publications

— Medel, P., Pournaghshband, V. (2017, March). Eliminating gender bias in computer science education materials. In Proceedings of the 2017 ACM SIGCSE technical symposium on computer science education (pp. 411-416). https://doi.org/10.1145/3017680.3017794

— Breidenbach, A., Mahlow, C., Schreiber, A. (2021). Implicit Gender Bias in Computer Science–A Qualitative Study. arXiv preprint arXiv :2107.01624. https://arxiv.org/pdf/2107.01624.pdf
— Edna, D. C., Tives, H. A., Madianita, B. M., Fagundes, F., José Antonio Siqueira, d. C. (2019). Barriers faced by women in software development projects. Information, 10(10), 309. http://dx.doi.org/10.3390/info10100309
— Gayna Williams. 2014. Are you sure your software is gender-neutral ? interactions 21, 1 (January - February 2014), 36–39. https://doi.org/10.1145/2524808
— Margaret Burnett, Simone Stumpf, Jamie Macbeth, Stephann Makri, Laura Beckwith, Irwin Kwan, Anicia Peters, William Jernigan, GenderMag : A Method for Evaluating Software's Gender Inclusiveness, Interacting with Computers, Volume 28, Issue 6, 19 November 2016, Pages 760–787. https://doi.org/10.1093/iwc/iwv046
— J. G. Stout and H. M. Wright, "Lesbian, Gay, Bisexual, Transgender, and Queer Students' Sense of Belonging in Computing : An Intersectional Approach," in Computing in Science and Engineering, vol. 18, no. 3, pp. 24-30, May-June 2016. https://doi.org/10.1109/MCSE.2016.45.

1.12 Green computing

The term *Green Computing* encompases an area of research seeking to decrease the environmental footprint of IT systems. It includes carbon footprint, water footprint, etc. of the design, implementation and exploitation of IT systems.

Examples of topics of interest

— Code energy consumption, the link between code complexity and energy consumption, benchmarking for energy consumption, etc.
— Code and implementation design for low energy consumption, energy antipatterns in mobile applications, energy antipatterns in web applications, etc.
— LLM water consumption footprint, LLM energy footprint, etc.

Relevant publications

— Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J. P., and Saraiva, J. (2021). Ranking programming languages by energy efficiency. Science of Computer Programming, 205, 102609. https://doi.org/10.1016/j.scico.2021.102609
— Bonvoisin, A., Quinton, C., and Rouvoy, R. (2024, March). Understanding the Performance-Energy Tradeoffs of Object-Relational Mapping Frameworks. In 31th IEEE International Conference on Software Analysis, Evolution and Reengineering-SANER 2024 (p. 11). IEEE. https://doi.org/10.1109/SANER60148.2024.00069
— Coignion, T., Quinton, C., and Rouvoy, R. (2024, June). A Performance Study of LLM-Generated Code on Leetcode. In Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (pp. 79-89). https://doi.org/10.1145/3661167.3661221
— Naumann, S., Dick, M., Kern, E., and Johann, T. (2011). The GREENSOFT Model : A reference model for green and sustainable software and its engineering. Sustainable Computing : Informatics and Systems, 1(4), 294-304. https://doi.org/10.1016/j.suscom.2011.06.004